



♦ UNIVERSITÉ ALIOUNE DIOP DE BAMBEY ♦

UFR : Sciences Appliquées et Technologies de l'Information et de la Communication

Département : Technologies de l'Information et de la Communication

Master 1 : Système d'Information

RAPPORT DE TRAVAUX PRATIQUES N°TP3

Création d'une application Web dans un conteneur Docker

Docker, Flask et automatisation Bash

PRÉSENTÉ PAR

Gana FAYE

DATE

15/05/2026

ANNÉE ACADÉMIQUE

2025 - 2026

Table des matières

Partie 1 : Lancer la machine virtuelle DEVASC	2
Partie 2 : Créer un script Bash simple	3
Partie 3 : Créer une application Web	5
Partie 4 : Configurer l'application Web	7
Partie 5 : Script Bash pour Docker	8
Partie 6 : Construire, exécuter et vérifier le conteneur	9
Conclusion	11

Partie 1 : Lancer la machine virtuelle DEVASC

Étape 1 : Démarrage de la VM

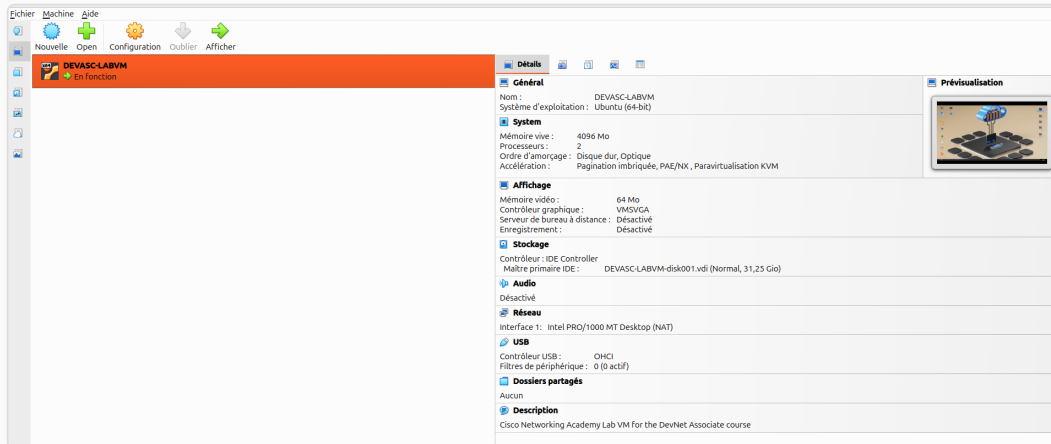


Figure 1.1 : Démarrage de la VM DEVASC

Étape 2 : Connexion à la VM

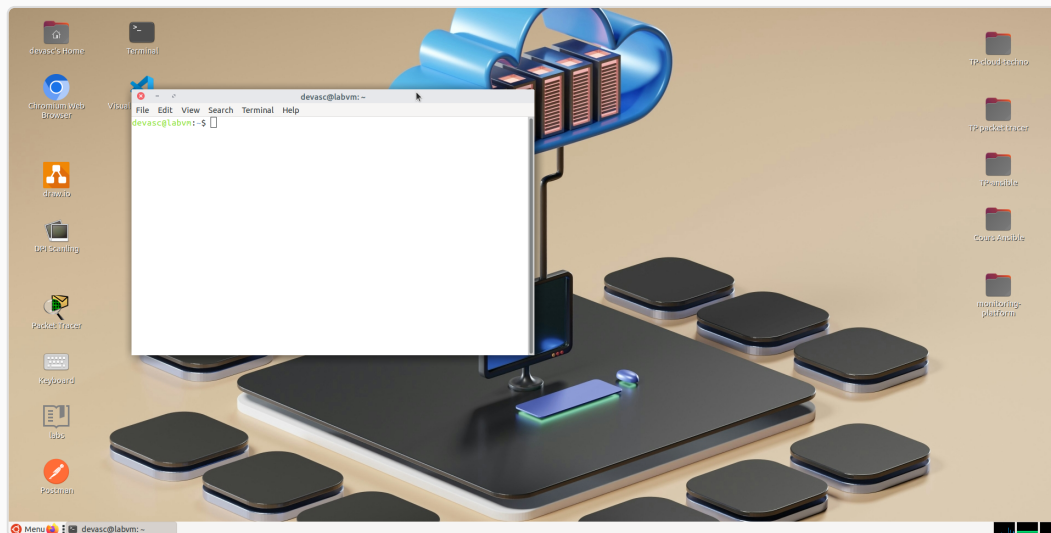


Figure 1.2 : Interface de connexion de la VM DEVASC

Étape 3 : Vérification du bureau

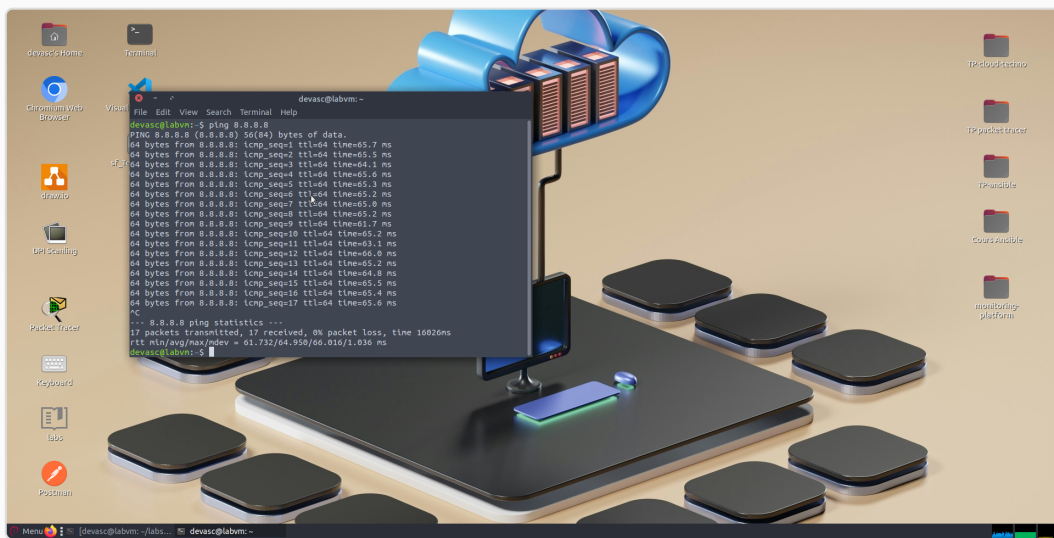


Figure 1.3 : Bureau de la VM DEVASC après connexion

Test de connectivité réseau

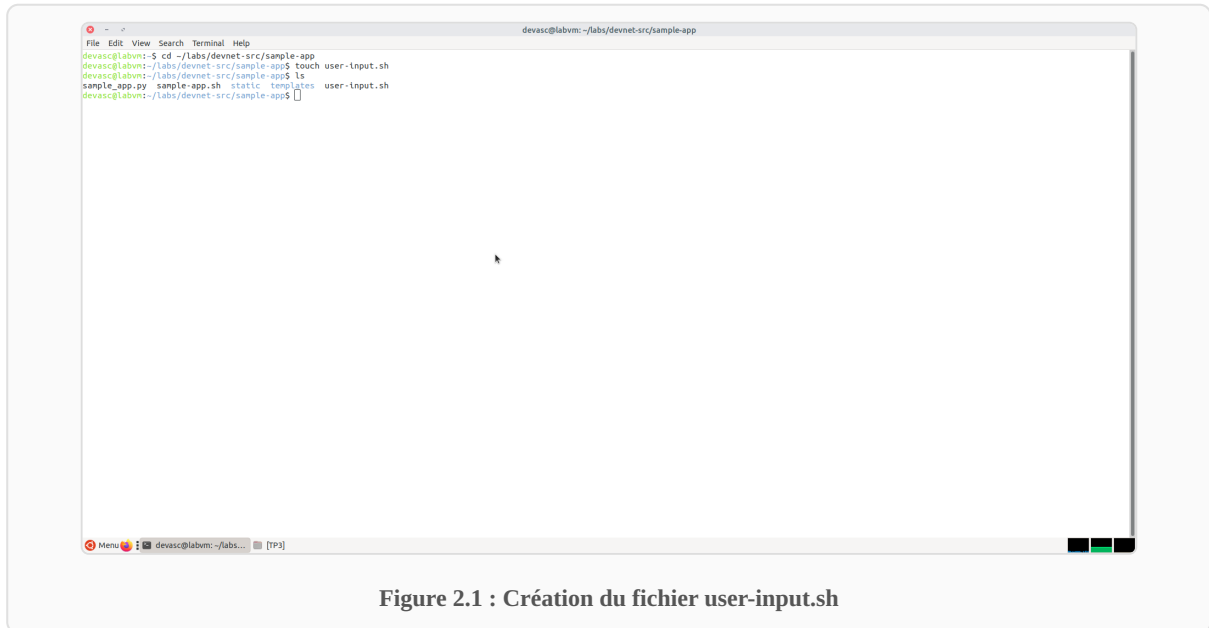
```
$ ping -c 4 google.com
```

🔍 Assurez-vous que la VM dispose d'un accès réseau fonctionnel avant de continuer.

Partie 2 : Créer un script Bash simple

Étape 1 : Créer un fichier de script bash vide

```
cd ~/labs/devnet-src/sample-app  
touch user-input.sh
```



Étapes 2 à 5 : Édition du script avec nano

```
nano user-input.sh  
  
#!/bin/bash  
echo -n "Enter Your Name: "  
read userName  
echo "Your name is $userName."
```

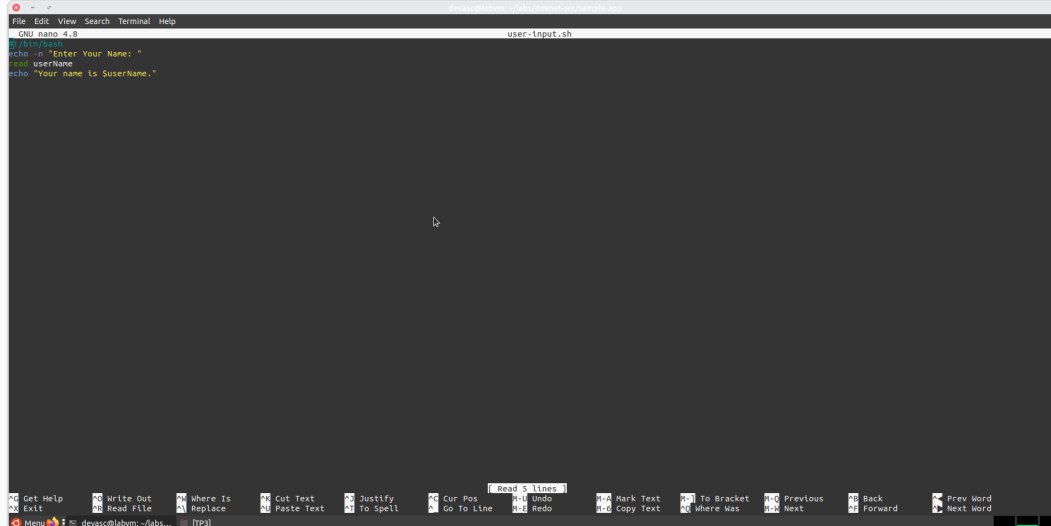


Figure 2.2 : Édition du script dans nano (étapes 2 à 5)

Étape 6 : Exécuter le script

```
bash user-input.sh
```

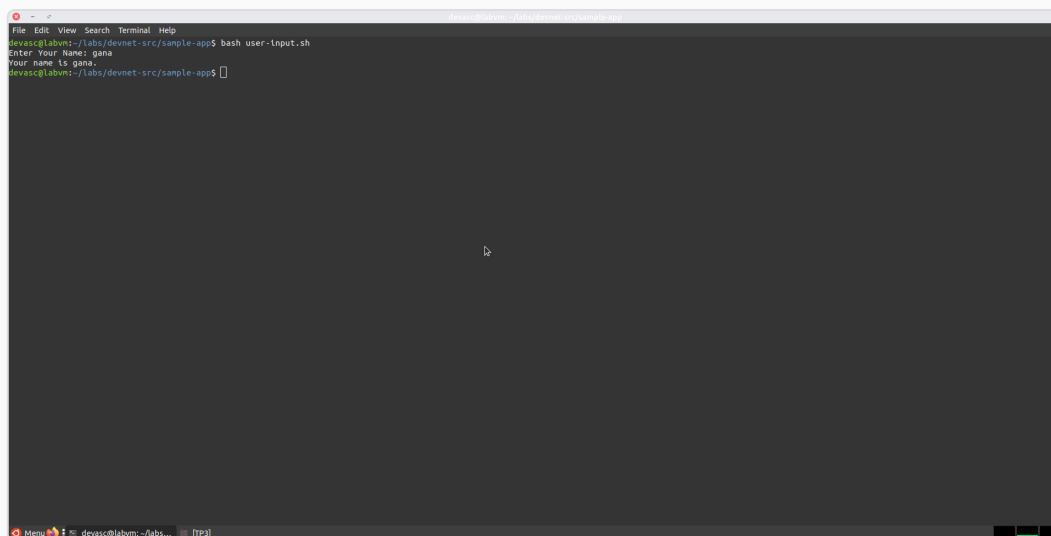
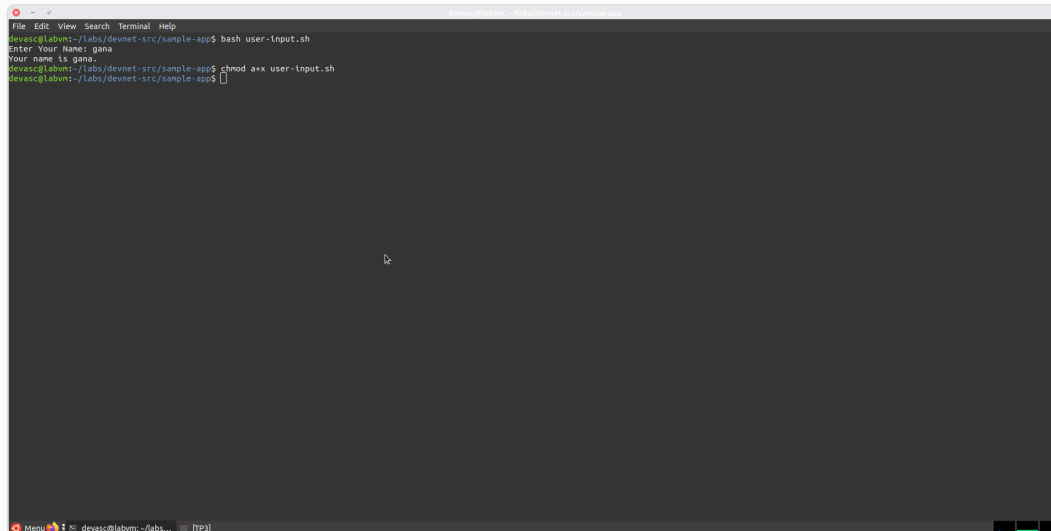


Figure 2.3 : Exécution du script bash user-input.sh

Étape 7 : Rendre le script exécutable

```
chmod a+x user-input.sh
```

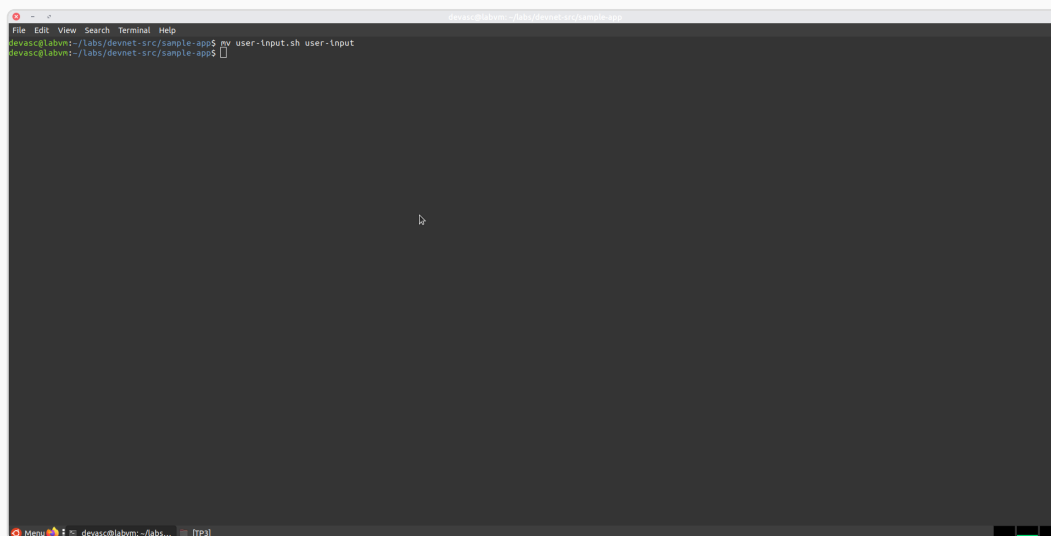
A terminal window titled "devasc@labvm: /labs/devnet-src/sample-app\$" showing the execution of a script. The prompt is "devasc@labvm:~/labs/devnet-src/sample-app\$". The user enters "bash user-input.sh". The script prompts "Enter Your Name: gena" and "Your name is gena.". The user then enters "chmod a+x user-input.sh". The prompt returns to "devasc@labvm:~/labs/devnet-src/sample-app\$".

```
devasc@labvm:~/labs/devnet-src/sample-app$ bash user-input.sh
Enter Your Name: gena
Your name is gena.
devasc@labvm:~/labs/devnet-src/sample-app$ chmod a+x user-input.sh
devasc@labvm:~/labs/devnet-src/sample-app$
```

Figure 2.4 : Commande chmod

Étape 8 : Renommer le fichier

```
mv user-input.sh user-input
```

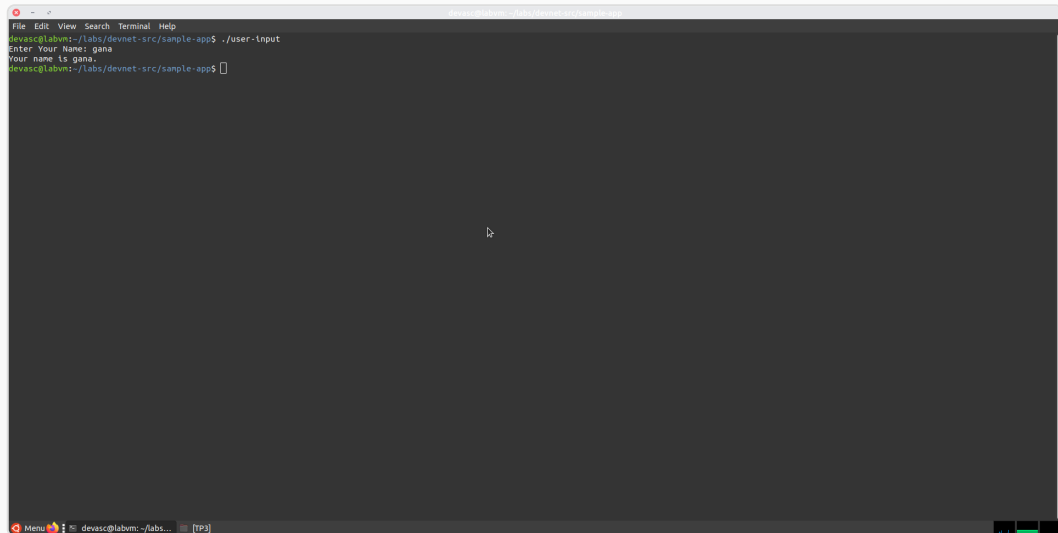
A terminal window titled "devasc@labvm: /labs/devnet-src/sample-app\$" showing the execution of a script. The prompt is "devasc@labvm:~/labs/devnet-src/sample-app\$". The user enters "mv user-input.sh user-input". The prompt returns to "devasc@labvm:~/labs/devnet-src/sample-app\$".

```
devasc@labvm:~/labs/devnet-src/sample-app$ mv user-input.sh user-input
devasc@labvm:~/labs/devnet-src/sample-app$
```

Figure 2.5 : Renommage du fichier

Étape 9 : Exécuter le script sans extension

```
./user-input
```



```
devasc@labvm:~/labs/devnet-src/sample-app$ ./user-input
Enter Your Name: gena
Your name is gena.
devasc@labvm:~/labs/devnet-src/sample-app$
```

Figure 2.6 : Exécution finale avec ./user-input

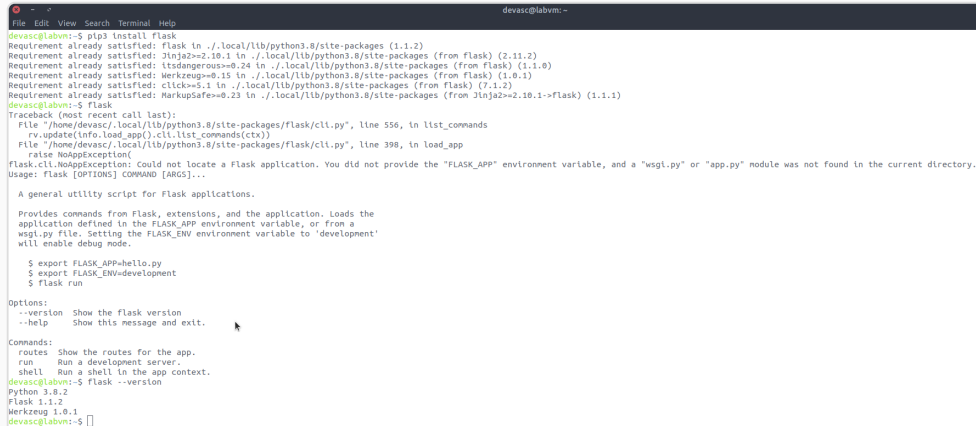
Étape 10 : Documentation

 Tutoriels Bash recommandés : tldp.org/LDP/abs/html/

Partie 3 : Créer un exemple d'application Web

Étape 1 : Installer Flask

```
pip3 install flask
```



```
devasc@labvm:~$ pip3 install Flask
Requirement already satisfied: flask in ./local/lib/python3.8/site-packages (1.1.2)
Requirement already satisfied: Jinja2>=2.10.1 in ./local/lib/python3.8/site-packages (from Flask) (2.11.2)
Requirement already satisfied: itsdangerous>=0.24 in ./local/lib/python3.8/site-packages (from Flask) (1.1.0)
Requirement already satisfied: Werkzeug>=0.15 in ./local/lib/python3.8/site-packages (from Flask) (1.0.1)
Requirement already satisfied: click>=5.1 in ./local/lib/python3.8/site-packages (from Flask) (7.1.2)
Requirement already satisfied: MarkupSafe>=0.23 in ./local/lib/python3.8/site-packages (from Jinja2>=2.10.1->flask) (1.1.1)
devasc@labvm:~$ flask
Traceback (most recent call last):
  File "/home/devasc/.local/lib/python3.8/site-packages/flask/cli.py", line 556, in list_commands
    rv.update(info.load_app().cli.list_commands(ctx))
  File "/home/devasc/.local/lib/python3.8/site-packages/flask/cli.py", line 398, in load_app
    raise NoAppException()
flask.cli.NoAppException: Could not locate a Flask application. You did not provide the "FLASK_APP" environment variable, and a "wsgi.py" or "app.py" module was not found in the current directory.
Usage: flask [OPTIONS] COMMAND [ARGS]...

A general utility script for Flask applications.

Provides commands from Flask, extensions, and the application. Loads the
application defined in the FLASK_APP environment variable, or from a
wsgi.py file. Setting the FLASK_ENV environment variable to 'development'
will enable debug mode.

$ export FLASK_APP=hello.py
$ export FLASK_ENV=development
$ flask run

Options:
  --version  show the flask version
  --help    show this message and exit.

Commands:
  routes  Show the routes for the app.
  run     Run a development server.
  shell   Run a shell in the app context.
python 3.8.2
Flask 1.1.2
Werkzeug 1.0.1
devasc@labvm:~$
```

Figure 3.1 : Installation de Flask

Étapes 3 à 6 : Code de l'application

```
from flask import Flask, request

sample = Flask(__name__)

@sample.route("/")

def main():

    return "You are calling me from " + request.remote_addr + "\n"

if __name__ == "__main__":

    sample.run(host="0.0.0.0", port=8080)
```

```

devasc@labvm: ~/labs/devnet-src/sample-app
GNU nano 4.8 sample_app.py
from flask import Flask
from flask import request

sample = Flask(__name__)

@sample.route("/")
def hello():
    # Cette ligne affiche l'IP de celui qui visite la page
    return "You are calling me from " + request.remote_addr + "\n"

if __name__ == "__main__":
    # Utilisation du port 8083 pour éviter le conflit avec Ansible port 8081 et prometheus 8082
    sample.run(host="0.0.0.0", port=8083)

```

Figure 3.2 : Code Flask (étapes 3 à 6)

Étape 7 : Exécuter l'application

```
python3 sample_app.py
```

```

devasc@labvm: ~/labs/devnet-src/sample-app
devasc@labvm:~/labs/devnet-src/sample-app$ nano sample_app.py
devasc@labvm:~/labs/devnet-src/sample-app$ python3 sample_app.py
* Serving Flask app "sample_app" (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
* Running on http://0.0.0.0:8083/ (Press CTRL+C to quit)

```

Figure 3.3 : Exécution de l'application Flask

Étape 8a : Vérification navigateur

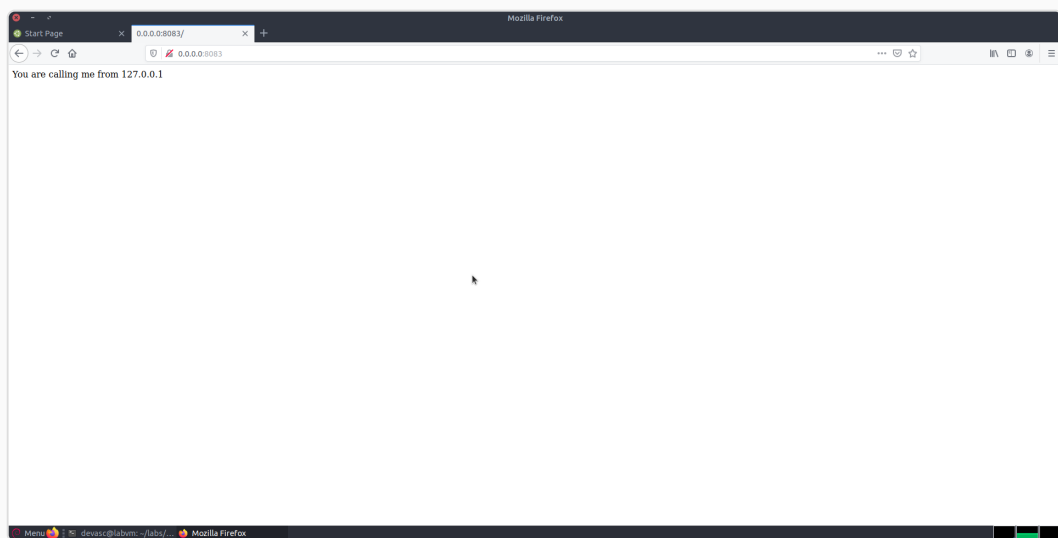


Figure 3.4 : Navigateur : 127.0.0.1

Étape 8b : Vérification cURL

```
curl http://0.0.0.0:8080
```

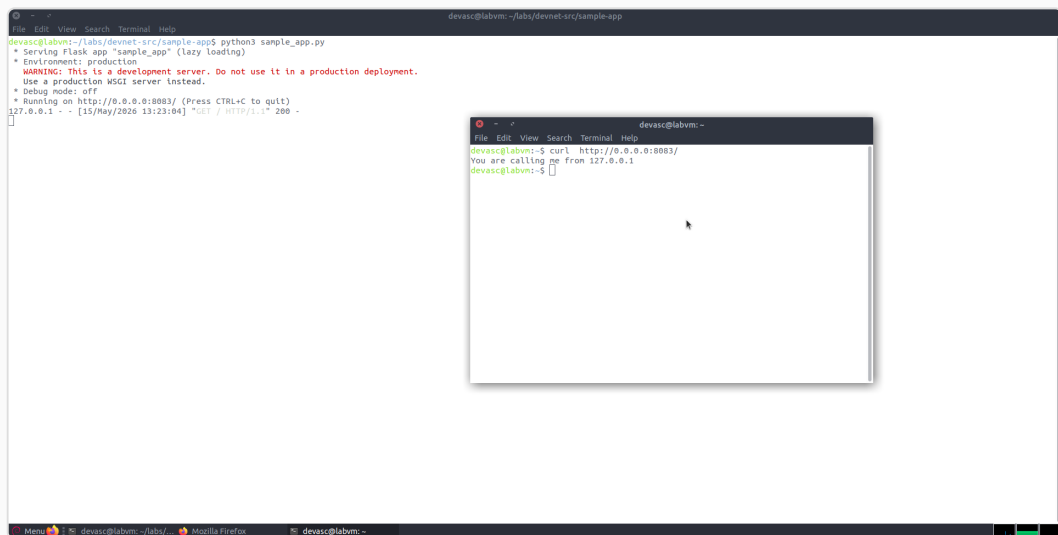
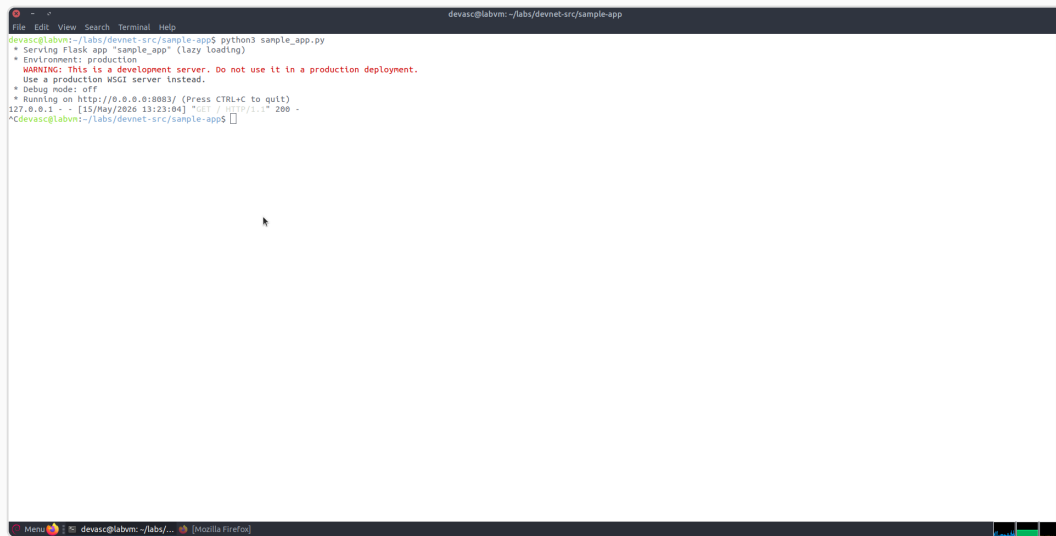


Figure 3.5 : Résultat de la commande curl

Étape 9 : Arrêter le serveur



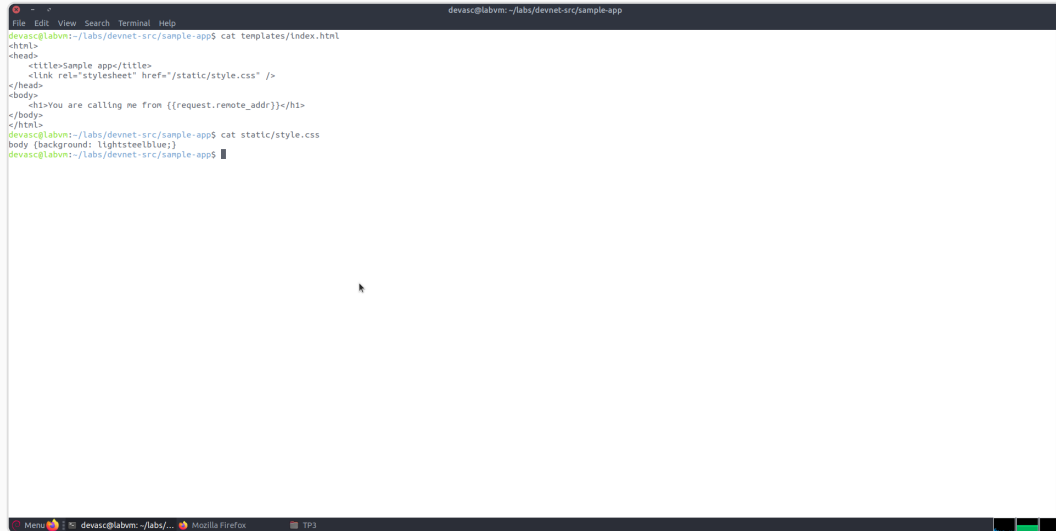
```
devasc@labvm: ~/labs/devnet-src/sample-app
devasc@labvm:~/labs/devnet-src/sample-app$ python3 sample_app.py
* Serving Flask app "sample_app" (lazy loading)
* Environment: production
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Debug mode: off
* Running on http://0.0.0.0:8080/ (Press CTRL+C to quit)
127.0.0.1 - - [15/May/2020 13:23:04] "GET / HTTP/1.1" 200
^Cdevasc@labvm:~/labs/devnet-src/sample-app$
```

Figure 3.6 : Arrêt du serveur avec CTRL+C

Partie 4 : Configurer l'application Web

Étape 1 : Explorer les répertoires

```
cat templates/index.html
cat static/style.css
```



The screenshot shows a terminal window with the following content:

```
devasc@labvm: ~/labs/devnet-src/sample-app
devasc@labvm:~/labs/devnet-src/sample-app$ cat templates/index.html
<html>
<head>
  <title>Sample app</title>
  <link rel="stylesheet" href="/static/style.css" />
</head>
<body>
  <h1>You are calling me from {{request.remote_addr}}</h1>
</body>
</html>
devasc@labvm:~/labs/devnet-src/sample-app$ cat static/style.css
body {background: lightsteelblue;}
devasc@labvm:~/labs/devnet-src/sample-app$
```

Figure 4.1 : Contenu de index.html et style.css

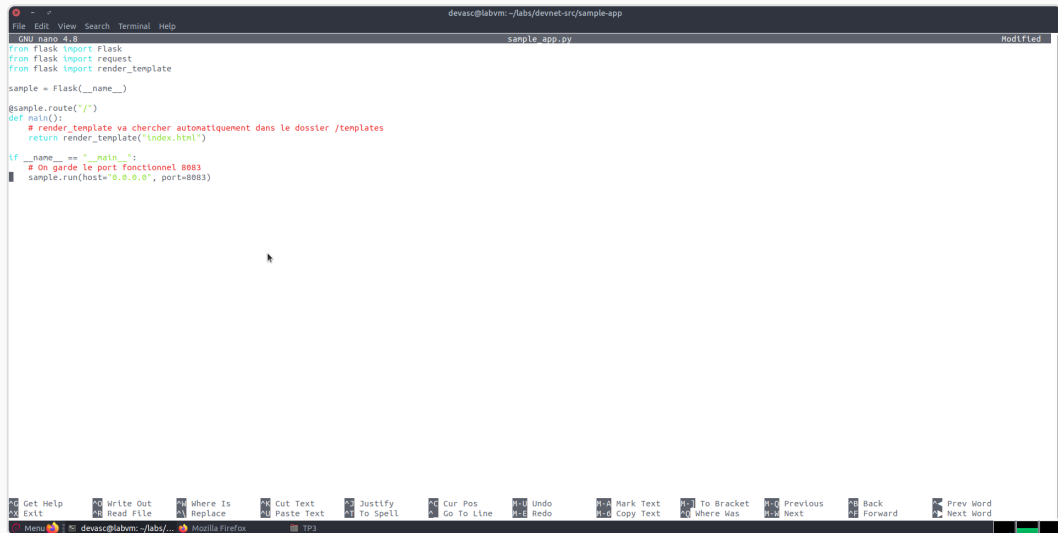
Étape 2 : Mettre à jour le code Python

```
from flask import render_template

@sample.route("/")

def main():

    return render_template("index.html")
```



```

devasc@labvm: ~/labs/devnet/src/sample-app
GNU nano 4.8 sample_app.py
from flask import Flask
from flask import request
from flask import render_template

sample = Flask(__name__)

@sample.route("/")
def main():
    # render_template va chercher automatiquement dans le dossier /templates
    return render_template("index.html")

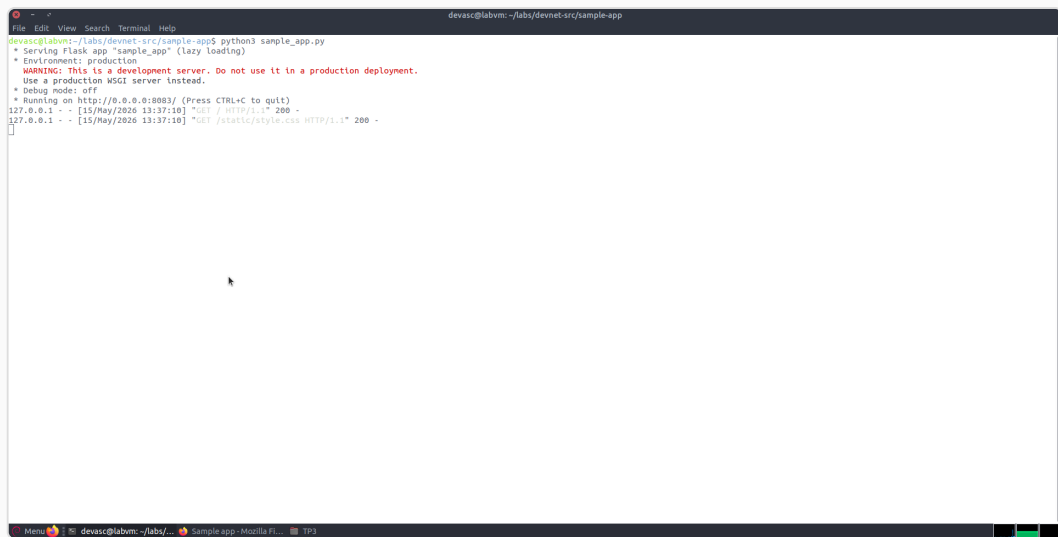
if __name__ == "__main__":
    # On garde le port fonctionnel 8083
    sample.run(host="0.0.0.0", port=8083)

```

Figure 4.2 : Code modifié avec render_template

Étape 3 : Exécuter le serveur

```
python3 sample_app.py
```



```

devasc@labvm: ~/labs/devnet/src/sample-app
devasc@labvm:~/labs/devnet/src/sample-app$ python3 sample_app.py
* Serving Flask app "sample_app" (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
* Running on http://0.0.0.0:8083/ (Press CTRL+C to quit)
127.0.0.1 - - [15/May/2026 13:37:10] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [15/May/2026 13:37:10] "GET /static/style.css HTTP/1.1" 200 -

```

Figure 4.3 : Exécution du serveur

Étape 4a : Vérification navigateur

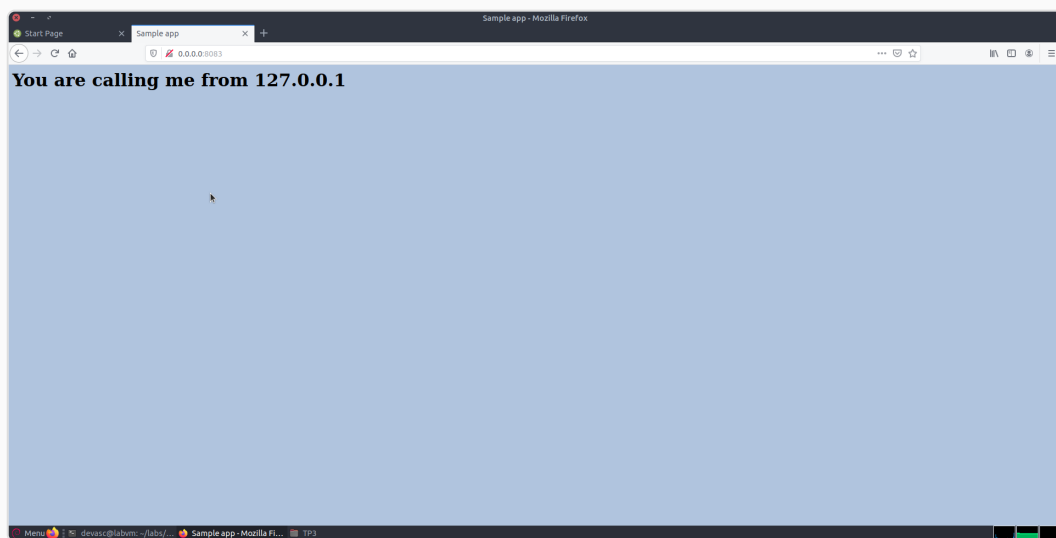


Figure 4.4 : Page web avec fond bleu acier

Étape 4b : Vérification cURL

```
curl http://0.0.0.0:8080
```

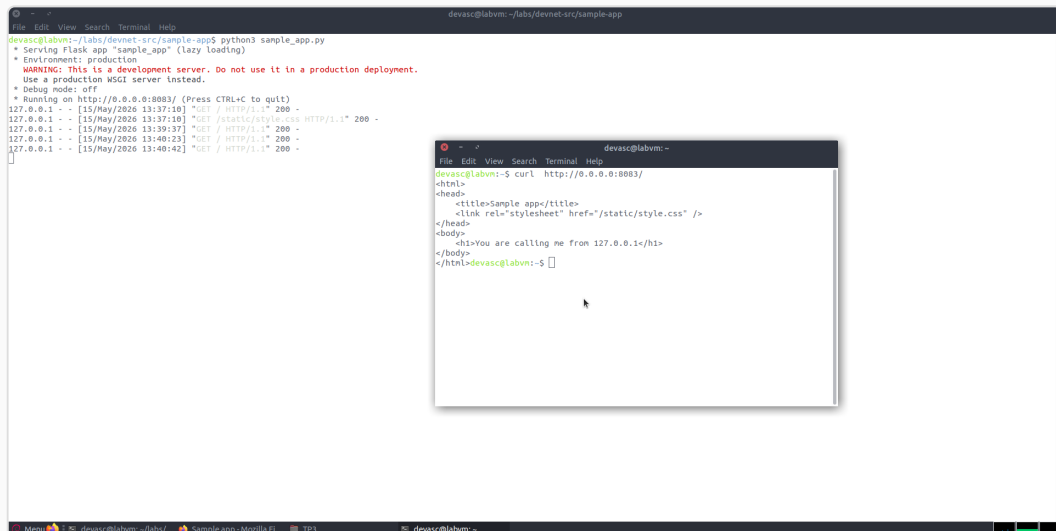


Figure 4.5 : Résultat curl avec HTML

Étape 5 : Arrêter le serveur



Figure 4.6 : Arrêt du serveur

Partie 5 : Créer un script Bash pour créer et exécuter un conteneur Docker

```
#!/bin/bash

mkdir tmpdir

mkdir tmpdir/templates

mkdir tmpdir/static

cp sample_app.py tmpdir/.

cp -r templates/* tmpdir/templates/.

cp -r static/* tmpdir/static/.

echo "FROM python" > tmpdir/Dockerfile

echo "RUN pip install flask" >> tmpdir/Dockerfile

echo "COPY ./static /home/myapp/static/" >> tmpdir/Dockerfile

echo "COPY ./templates /home/myapp/templates/" >> tmpdir/Dockerfile

echo "COPY sample_app.py /home/myapp/" >> tmpdir/Dockerfile

echo "EXPOSE 8080" >> tmpdir/Dockerfile

echo "CMD python3 /home/myapp/sample_app.py" >> tmpdir/Dockerfile

cd tmpdir

docker build -t sampleapp .

docker run -t -d -p 8080:8080 --name samplerunning sampleapp

docker ps -a
```

```

devasc@labvm: ~/labs/devnet-src/sample-app
GNU nano 4.8 sample-app.sh

#!/bin/bash

# 1. Nettoyage des dossiers et creation de la structure
rm -rf tmpdir
mkdir -p tmpdir/templates
mkdir -p tmpdir/static

# 2. Copie des fichiers sources
cp sample_app.py tmpdir/.
cp -r templates/* tmpdir/templates/.
cp -r static/* tmpdir/static/.

# 3. Generation du Dockerfile
echo "FROM python:3.9-slim" > tmpdir/Dockerfile
echo "RUN pip install --no-cache-dir --progress-bar off flask" >> tmpdir/Dockerfile

# 4. On expose le port 8083 car c'est celui configure dans ton sample_app.py
echo "WORKDIR /home/myapp/" > tmpdir/Dockerfile
echo "COPY ./static /home/myapp/static/" >> tmpdir/Dockerfile
echo "COPY ./templates /home/myapp/templates/" >> tmpdir/Dockerfile
echo "COPY sample_app.py /home/myapp/" >> tmpdir/Dockerfile

# 5. On expose le port 8083 car c'est celui configure dans ton sample_app.py
echo "EXPOSE 8083" > tmpdir/Dockerfile
echo "CMD ["python", "sample_app.py"]" >> tmpdir/Dockerfile

# 6. Construction de l'image
cd tmpdir
docker rm -f sampleapp 2 /dev/null
docker build -t sampleapp .

# 7. Nettoyage et lancement du conteneur
cd ..
docker stop samplerunning 2 /dev/null
docker rm -f samplerunning 2 /dev/null

# 8. Lancement du conteneur
docker run -t -d -p 8084:8083 --name samplerunning sampleapp

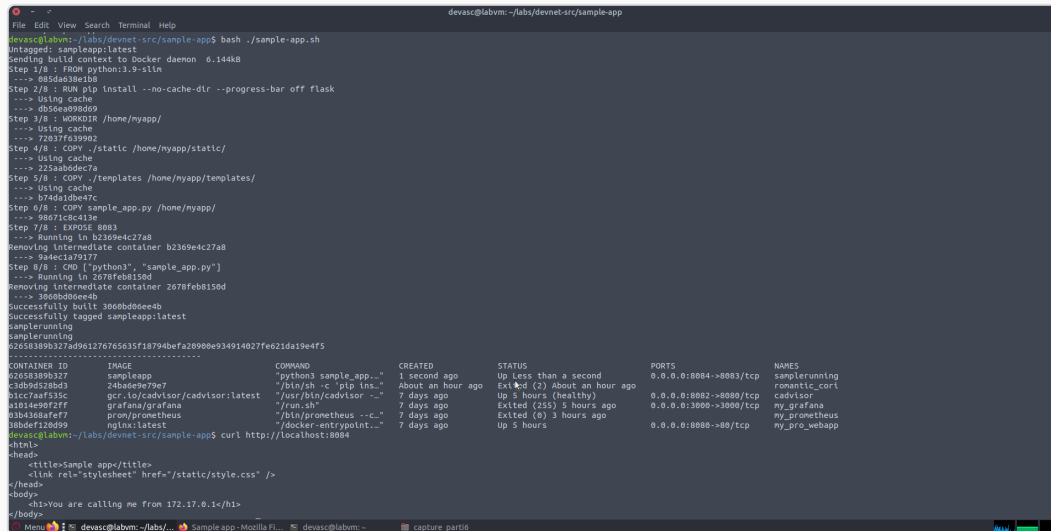
# 9. Verification
docker ps -l

```

Figure 5.1 : Script sample-app.sh

Partie 6 : Construire, exécuter et vérifier le conteneur Docker

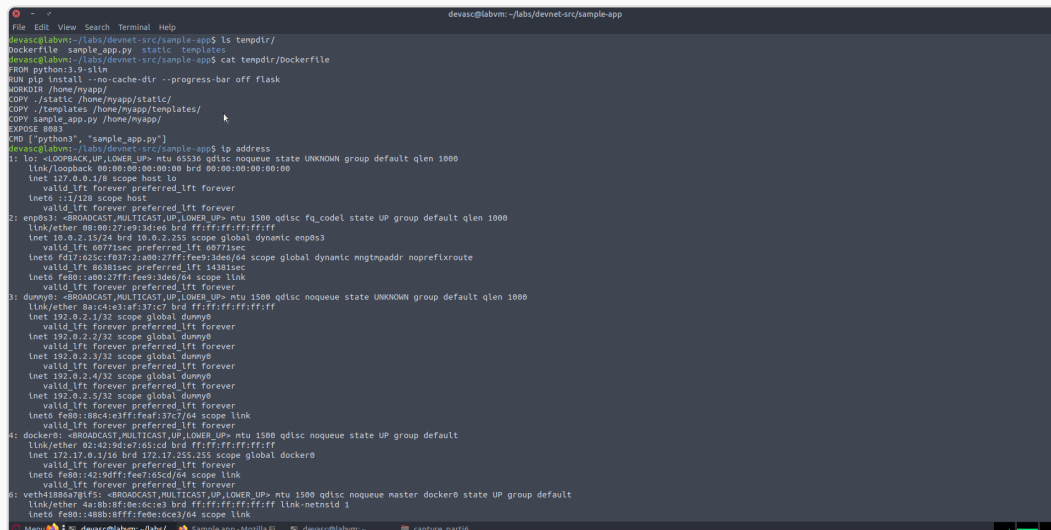
Étape 1 : Création du script



```
devasc@labvm: ~/labs/devnet-src/sample-app
File Edit View Search Terminal Help
devasc@labvm:~/labs/devnet-src/sample-app$ bash ./sample-app.sh
Intagged: sampleapp:latest
Sending build context to Docker daemon 6.144kB
Step 1/8 : FROM python:3.9-slim
--> 445da33a118e
Step 2/8 : RUN pip install --no-cache-dir --progress-bar off flask
--> Using cache
--> 465ea99d99
Step 3/8 : WORKDIR /home/myapp/
--> Using cache
--> 72037f639902
Step 4/8 : COPY ./static /home/myapp/static/
--> Using cache
--> 225aab0dec7a
Step 5/8 : COPY ./templates /home/myapp/templates/
--> Using cache
--> b74d310be47c
Step 6/8 : COPY sample_app.py /home/myapp/
--> 98071c8c413e
Step 7/8 : EXPOSE 8083
--> Running in b23694c27a8
Removing intermediate container b23694c27a8
--> 94e4c139127
Step 8/8 : CMD ["python3", "sample_app.py"]
--> Running in 2678feb150d
Removing intermediate container 2678feb150d
--> 3060bd0ee4b
Successfully built 3060bd0ee4b
Successfully tagged sampleapp:latest
samplerunning
CONTAINER ID        IMAGE               COMMAND                  CREATED              STATUS              PORTS              NAMES
62658389b327ad961276765635f18794befa2090e934914027fe021da19e4f5 sampleapp            "python3 sample_app..." 1 second ago        Up less than a second    0.0.0.0:8084->8083/tcp    samplerunning
23d8d6d230d83       24b0e9e79e7         "/bin/sh -c 'pip ins..." About an hour ago    Exited (2) About an hour ago    0.0.0.0:8082->8080/tcp    romantic_cort
b1cc7aa7533c        gcr.io/cadvisor/cadvisor:latest "/usr/bin/cadvisor ..." 7 days ago          Up 5 hours (healthy)    0.0.0.0:8080->8080/tcp    cadvisor
d1014e90f2ff        grafana/grafana     "/run.sh"               7 days ago          Exited (255) 5 hours ago    0.0.0.0:3000->3000/tcp    my_grafana
b3b33d4ef77         prom/prometheus     "/bin/prometheus -c..." 7 days ago          Exited (6) 3 hours ago    0.0.0.0:8088->80/tcp      my_prometheus
8b8def120d99        nginx:latest        "/docker-entrypoint..." 7 days ago          Up 5 hours              0.0.0.0:8088->80/tcp      my_pro_webapp
devasc@labvm:~/labs/devnet-src/sample-app$ curl http://localhost:8084
<html>
<head>
<title>Sample app</title>
<link rel="stylesheet" href="/static/style.css" />
</head>
<body>
<h1>You are calling me from 172.17.0.14</h1>
</body>
</html>
```

Figure 6.1 : Création du script sample-app.sh

Étape 2 : Exécution (création répertoires)



```
devasc@labvm: ~/labs/devnet-src/sample-app
File Edit View Search Terminal Help
devasc@labvm:~/labs/devnet-src/sample-app$ ls tmpdir/
Dockerfile sample_app.py static templates
devasc@labvm:~/labs/devnet-src/sample-app$ cat tmpdir/Dockerfile
FROM python:3.9-slim
RUN pip install --no-cache-dir --progress-bar off flask
WORKDIR /home/myapp/
COPY ./static /home/myapp/static/
COPY ./templates /home/myapp/templates/
COPY sample_app.py /home/myapp/
EXPOSE 8083
CMD ["python3", "sample_app.py"]
devasc@labvm:~/labs/devnet-src/sample-app$ ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:e9:3d:40 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic enp0s3
        valid_lft 6071sec preferred_lft 6071sec
    inet6 fd17:025c:f037:2:a00:27fff:fe9:3de0/64 scope global dynamic mngtmpaddr nopreflxroute
        valid_lft 80381sec preferred_lft 14381sec
    inet6 fe8b::a00:27fff:fe9:3de0/64 scope link
        valid_lft forever preferred_lft forever
3: dummy0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN group default qlen 1000
    link/ether Ba:ca:e3:af:37:c7 brd ff:ff:ff:ff:ff:ff
    inet 192.0.2.1/32 scope global dummy0
        valid_lft forever preferred_lft forever
    inet 192.0.2.2/32 scope global dummy0
        valid_lft forever preferred_lft forever
    inet 192.0.2.3/32 scope global dummy0
        valid_lft forever preferred_lft forever
    inet 192.0.2.4/32 scope global dummy0
        valid_lft forever preferred_lft forever
    inet 192.0.2.5/32 scope global dummy0
        valid_lft forever preferred_lft forever
    inet6 fe8b::8bc4:e3ff:feaf:37c7/64 scope link
        valid_lft forever preferred_lft forever
4: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:9d:7e:05:cd brd ffff:ffff:ffff:ffff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
    inet6 fe8b::d2:9fff:fe7:f5dc/64 scope link
        valid_lft forever preferred_lft forever
    inet6 fe8b::d2:9fff:fe7:f5dc/64 scope link
        valid_lft forever preferred_lft forever
5: veth4188a78151: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group default
    link/ether 4a:8b:5f:de:dc:63 brd ffff:ffff:ffff:ffff link-netnsid 1
    inet6 fe8b::48bb:fefc:de3/64 scope link
```

Figure 6.2 : Création des répertoires tmpdir

Étape 3 : Exécution (copie fichiers)

```
devasc@labvm: ~/labs/devnet-src/sample-app
devasc@labvm:~/labs/devnet-src/sample-app$ bash ./sample-app.sh
Untagged: sampleapp:latest
Sending build context to Docker daemon  6.144kB
Step 1/8 : FROM python:3.9-slim
--> 485a633e1e1b
Step 2/8 : RUN pip install --no-cache-dir --progress-bar off flask
--> Using cache
--> 435eab8b0d09
Step 3/8 : WORKDIR /home/myapp/
--> Using cache
--> 72831f6399a2
Step 4/8 : COPY ./static /home/myapp/static/
--> Using cache
--> 225a8b6dec7a
Step 5/8 : COPY ./templates /home/myapp/templates/
--> Using cache
--> b74de1de47c
Step 6/8 : COPY sample_app.py /home/myapp/
--> 58071c8c41e
Step 7/8 : EXPOSE 8083
--> Running in b269e4c27a8
Removing intermediate container b269e4c27a8
Step 8/8 : CMD ["python3", "sample_app.py"]
--> Running in 2678feb159d
Removing intermediate container 2678feb159d
--> 306b0bd0eeab
Successfully built 306b0bd0eeab
Successfully tagged sampleapp:latest
samplerrunning
62558389b327ad951276765635f18794befa2090e934914027fe621da19e4f5
-----
CONTAINER ID        IMAGE               COMMAND                  CREATED              STATUS              PORTS              NAMES
62558389b327       sampleapp           "python3 sample_app..." 1 second ago        Up Less than a second    0.0.0.0:8084->8083/tcp    samplerrunning
3db9d528bd3       24ba6de7e7e7       "/bin/sh -c 'pip ins..." About an hour ago    Exited (2) About an hour ago          0.0.0.0:8082->8080/tcp    romantic_cort
51cc7ae4333c       gcr.io/cadvisior/cadvisior:latest "usr/bin/cadvisior -" 7 days ago          Up 5 hours (healthy)    0.0.0.0:8082->8080/tcp    cadvisior
401d4e90f2ff       grafana/grafana     "run.sh"                 7 days ago          Exited (255) 5 hours ago          0.0.0.0:3000->3000/tcp    my_grafana
2b3a3a3aef7       prom/prometheus -c-" /bin/prometheus -c..." 7 days ago          Exited (6) 3 hours ago          0.0.0.0:8088->80/tcp      my_prometheus
8b0ef120d99       nginx:latest        "docker-entrypoint..." 7 days ago          Up 5 hours              0.0.0.0:8088->80/tcp      my_pro_webapp

devasc@labvm:~/labs/devnet-src/sample-app$ curl http://localhost:8084
<html>
<title>Sample app</title>
<link rel="stylesheet" href="/static/style.css" />
</head>
<body>
<h1>You are calling me from 172.17.0.1</h1>
</body>
</html>
```

Figure 6.3 : Copie des fichiers vers tempdir

Étape 4 : Construction de l'image

```
devasc@labvm: ~/labs/devnet-src/sample-app
devasc@labvm:~/labs/devnet-src/sample-app$ CMD ["python3", "sample_app.py"]
devasc@labvm:~/labs/devnet-src/sample-app$ ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet ::1/128 scope host
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:ea:3d:e0 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic enp0s3
        valid_lft 6071sec preferred_lft 6071sec
    inet6 fd17:025c:f037:2:a00:27fff:fe9:3de6/64 scope global dynamic mngtmpaddr nopreflrout
        valid_lft 86381sec preferred_lft 14381sec
    inet6 fe80::a00:27fff:fe9:3de6/64 scope link
    valid_lft forever preferred_lft forever
3: dummy0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN group default qlen 1000
    link/ether Ba:ca:e3:af:37:c7 brd ff:ff:ff:ff:ff:ff
    inet 192.0.2.1/32 scope global dummy0
        valid_lft forever preferred_lft forever
    inet 192.0.2.2/32 scope global dummy0
        valid_lft forever preferred_lft forever
    inet 192.0.2.3/32 scope global dummy0
        valid_lft forever preferred_lft forever
    inet 192.0.2.4/32 scope global dummy0
        valid_lft forever preferred_lft forever
    inet 192.0.2.5/32 scope global dummy0
        valid_lft forever preferred_lft forever
    inet6 fe80::88c4:e3ff:feaf:37c7/64 scope link
        valid_lft forever preferred_lft forever
4: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:9d:ae:7d:cd brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
    inet6 fe80::42:9d:fe:7d:cd/64 scope link
        valid_lft forever preferred_lft forever
5: veth4188a6781f5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group default
    link/ether 4a:8b:1f:8e:dc:e3 brd ff:ff:ff:ff:ff:ff link-netnsid 1
    inet6 fe80::48b:1fff:fe8e:dc63/64 scope link
        valid_lft forever preferred_lft forever
6: veth50271e8177: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group default
    link/ether 1e:4c:5c:28:fa:22 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet6 fe80::5c4c:5c28:fa22:00/64 scope link
        valid_lft forever preferred_lft forever
7: veth4d7bc2281f21: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group default
    link/ether 90:5e:98:c3:45:08 brd ff:ff:ff:ff:ff:ff link-netnsid 2
    inet6 fe80::945e:98ff:ec3:4508/64 scope link
        valid_lft forever preferred_lft forever
```

Figure 6.4 : Construction de l'image Docker

Étape 5 : Lancement du conteneur

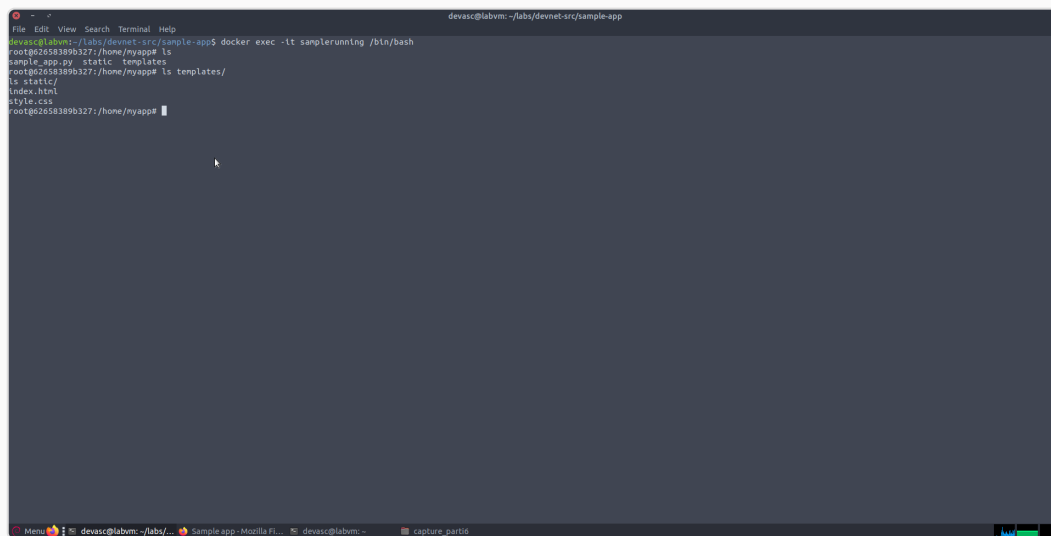


Figure 6.5 : Lancement du conteneur Docker

Étape 6 : Vérification

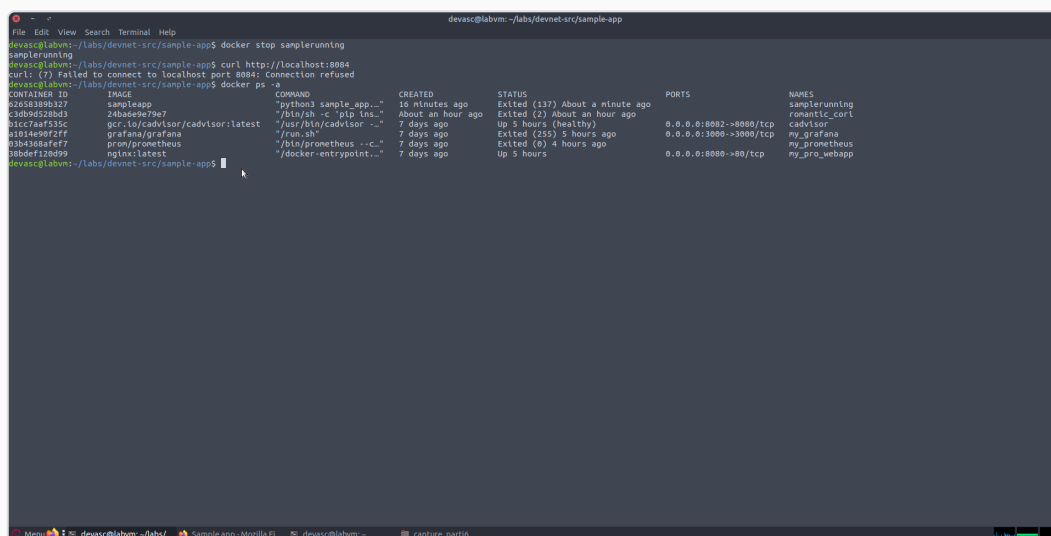


Figure 6.6 : Conteneur en cours d'exécution

Étape 7 : Test navigateur

 CAPTURE MANQUANTE
Navigateur : 172.17.0.1

Figure 6.7 : Navigateur : 172.17.0.1

Étape 8 : Suppression

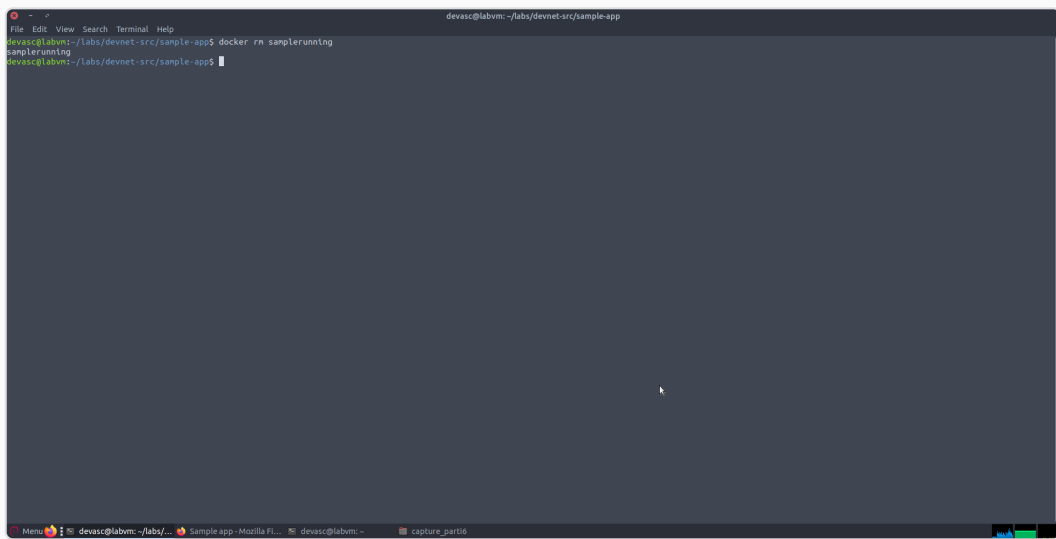


Figure 6.8 : Suppression des conteneurs

Étape 9 : Résultat final

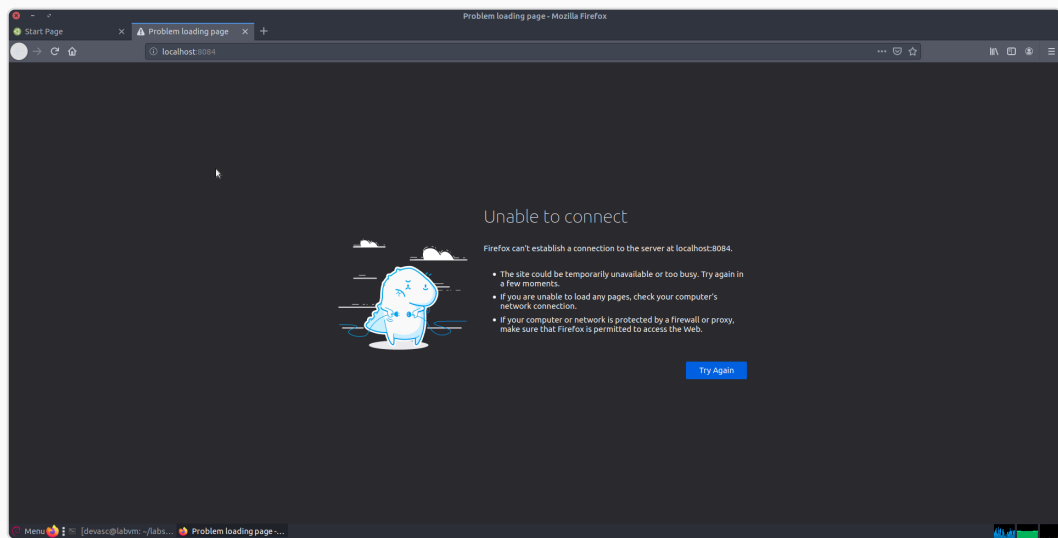


Figure 6.9 : État final des conteneurs

Commandes Docker essentielles :

`docker build -t nom .` | `docker run -d -p port:port nom` | `docker ps -a` | `docker stop nom` | `docker rm nom`

Conclusion

Ce travail pratique a permis de maîtriser l'ensemble du processus de conteneurisation d'une application web avec Docker.

- ✓ Création et exécution de scripts Bash
- ✓ Développement d'une application web avec Flask
- ✓ Intégration de templates HTML et CSS
- ✓ Création de Dockerfiles
- ✓ Construction, exécution et gestion de conteneurs Docker